

contents

01 인공지능 / 데이터 분석

- Pandas 이해

- Series : 1차원 구조
- DataFrame : 2차원 구조
- Pannel : 3차원 구조

- 데이터 입출력

- 데이터 조작

- Numpy 이해와 적용

- Numpy 실습

Pandas

- ① Pandas : 데이터분석용 라이브러리로 데이터를 다루는 패키지 중 하나임
- ② 데이터 분석 및 조작을 위한 효율적인 데이터 구조를 제공함
 - 행과 열로 이루어진 데이터 객체를 다룸
 - 안정적으로 대용량 데이터들을 처리함
 - 누락된 데이터를 유연하게 처리할 수 있는 기능 제공함

Pandas의 자료구조

- Series: 1차원 자료 구조
- DataFrame: 2차원 자료구조
- Pannel: 3차원 자료구조

```
import pandas as pd
```

Pandas 에서 사용되는 대표적인 데이터 오브젝트

시리즈 (Series)

Series 는 1차원 배열의 형태를 갖는다.
인덱스(노란색)라는 한 가지 기준에
의하여 데이터가 저장된다.

데이터프레임 (DataFrame)

DataFrame 은 2차원 배열의 형태를 갖는다.
인덱스(노란색)와 컬럼(파란색)이라는 두 가지
기준에 의하여 표 형태처럼 데이터가 저장된다.

데이터입출력 : Pandas로 데이터 읽기 / 저장

Pandas-Series

① Series 특성

- 1차원으로 되어 있음
- value와 index의 형태를 가지는 자료구조

입력값: 리스트(List) or 딕셔너리(Dictionary) 같은 Sequence 형태의 데이터

각 항목값(value)에 대응되는 색인(index)으로 구성
별도의 색인값이 없으면 0부터 시작

- List의 슬라이싱과 동일하게 적용 [start:end]
- Index 값/번호를 기준으로 슬라이싱 수행

```
1 import pandas as pd #pandas 패키지를 pd라는 명칭으로 축약해 호출
2 import numpy as np #numpy 패키지를 np라는 명칭으로 축약해 호출
3
4 #Series의 Value와 Index를 변수 pandas_series에 선언
5 pandas_series = pd.Series([3000, 3200, 2700],
6                             index = ['2016-11-10', '2016-11-11', '2016-11-12'])
7
8 print(type(pandas_series)) #변수 pandas_series의 type 출력
9 pandas_series              #변수 pandas_series 실행
```

<class 'pandas.core.series.Series'>

```
2016-11-10    3000
2016-11-11    3200
2016-11-12    2700
dtype: int64
```

Index	Value
2016-11-10	3000
2016-11-11	3200
2016-11-12	2700

데이터입출력 : Pandas로 데이터 읽기 / 저장

Pandas-Series : Dictionary 적용

```
import pandas as pd

menu = {"커피": 2000, "라떼": 2500, "그린티": 3000, "카페모카": 3500, "카푸치노": 4000}
se = pd.Series(menu)
print(se)
```

```
커피      2000
라떼      2500
그린티    3000
카페모카  3500
카푸치노  4000
dtype: int64
```

```
print(se.index)
print(se.values)
```

```
Index(['커피', '라떼', '그린티', '카페모카', '카푸치노'], dtype='object')
[2000 2500 3000 3500 4000]
```

index value

커피	2000
라떼	2500
그린티	3000
카페모카	3500
카푸치노	4000

```
Index(['커피', '라떼', '그린티', '카페모카', '카푸치노'], dtype='object')
[2000 2500 3000 3500 4000]
```

데이터입출력 : Pandas로 데이터 읽기 / 저장

Pandas-DataFrame

① DataFrame 생성

```
1 import pandas as pd #pandas 패키지를 pd라는 명칭으로 축약해 호출
2 import numpy as np #numpy 패키지를 np라는 명칭으로 축약해 호출
3
4 #DataFrame의 Value, Column, Index를 변수 df에 선언
5 df = pd.DataFrame([100, 150, 200, 250, 300], columns = ['numbers'],
6                 index = ['a', 'b', 'c', 'd', 'e'])
7 df
```

	value
	numbers
key	a 100
	b 150
	c 200
	d 250
	e 300

```
df.sum() #DataFrame의 Value를 모두 더함
```

```
numbers    1000
dtype: int64
```

```
df.numbers ** 2 #DataFrame의 numbers 칼럼의 각 값을 제곱
```

```
a    10000
b    22500
c    40000
d    62500
e    90000
```

```
Name: numbers, dtype: int64
```

데이터입출력 : Pandas로 데이터 읽기 / 저장

Pandas-DataFrame 관련 함수

종류	설명
sum()	행, 열 전체 성분의 합을 계산
mean()	행, 열 전체 성분의 평균을 계산
min(), max()	행, 열 전체 성분의 최소값, 최대값을 계산
count()	행, 열 전체 성분의(NaN이 아닌) 값의 개수를 계산
std(), var()	행, 열 전체 성분의 표준편차, 분산을 계산

	numbers
a	100
b	150
c	200
d	250
e	300

```
1 #DataFrame의 values라는 컬럼을 추가하고 값을 입력
2 df['values'] = (10, 50, 40, 30, 60)
3 df
```

	numbers	values
a	100	10
b	150	50
c	200	40
d	250	30
e	300	60

```
1 #DataFrame의 index의 값에 일치하는 값을 넣음
2 df['values'] = pd.DataFrame(['Fourth', 'Second', 'First', 'Fifth', 'Third'],
3                             index = ['d', 'b', 'a', 'e', 'c'])
4 df
```

	numbers	values
a	100	First
b	150	Second
c	200	Third
d	250	Fourth
e	300	Fifth

데이터입출력 : Pandas로 데이터 읽기 / 저장

Pandas-DataFrame 을 이용한 연산

종류	설명
sum()	행, 열 전체 성분의 합을 계산
mean()	행, 열 전체 성분의 평균을 계산
min(), max()	행, 열 전체 성분의 최소값, 최대값을 계산
count()	행, 열 전체 성분의(NaN이 아닌) 값의 개수를 계산
std(), var()	행, 열 전체 성분의 표준편차, 분산을 계산

```

1 import pandas as pd
2 import numpy as np
3
4 # random값을 넣은 DataFrame을 생성
5 # np.random.randn은 5행 5열의 랜덤 데이터를 생성
6 # 재호출 시 데이터가 변형되므로 주의
7 df=pd.DataFrame(np.random.randn(5,5))
8 df.columns=['A','B','C','D','E']
9
10 df

```

	A	B	C	D	E
0	-0.115540	-0.023043	0.994561	1.523005	0.623538
1	1.292809	-0.408988	-0.990792	-0.729662	-1.575598
2	0.944633	0.031914	0.492421	1.302999	0.527092
3	0.064526	-2.138368	-1.260443	-0.059801	0.911413
4	-0.094616	-1.721012	0.617084	0.722013	0.256642

각 Column별 평균값 출력
df.mean()

```

A    0.240514
B    0.225153
C   -0.462852
D   -0.457487
E    0.381158
dtype: float64

```

각 Column별 표준편차 값 출력
df.std()

```

A    1.976495
B    1.458724
C    1.223023
D    0.668688
E    0.482414
dtype: float64

```

- 함수의 인수 안에 axis = 0 (열) 또는 axis = 1(행)을 입력하면 행의 평균값, 표준편차를 얻을 수 있음

데이터입출력 : Pandas로 데이터 읽기 / 저장

① Pandas-DataFrame : 통계적분포확인

- shape 함수 : DataFrame 크기(행, 열)
- info 함수 : DataFrame의 기본 정보 확인

Describe 함수를 이용해 DataFrame 데이터의 통계 요약값 표현
df.describe()

	A	B	C	D	E
count	5.000000	5.000000	5.000000	5.000000	5.000000
mean	0.240514	0.225153	-0.462852	-0.457487	0.381158
std	1.976495	1.458724	1.223023	0.668688	0.482414
min	-2.748275	-1.093980	-1.936413	-1.206294	-0.155345
25%	-0.392381	-0.465369	-1.361758	-0.848546	-0.072795
50%	0.332449	-0.053895	-0.254194	-0.692138	0.447916
75%	1.840015	0.028305	0.062654	0.035831	0.788594
max	2.170760	2.710702	1.175449	0.423712	0.897418

Numpy 이해

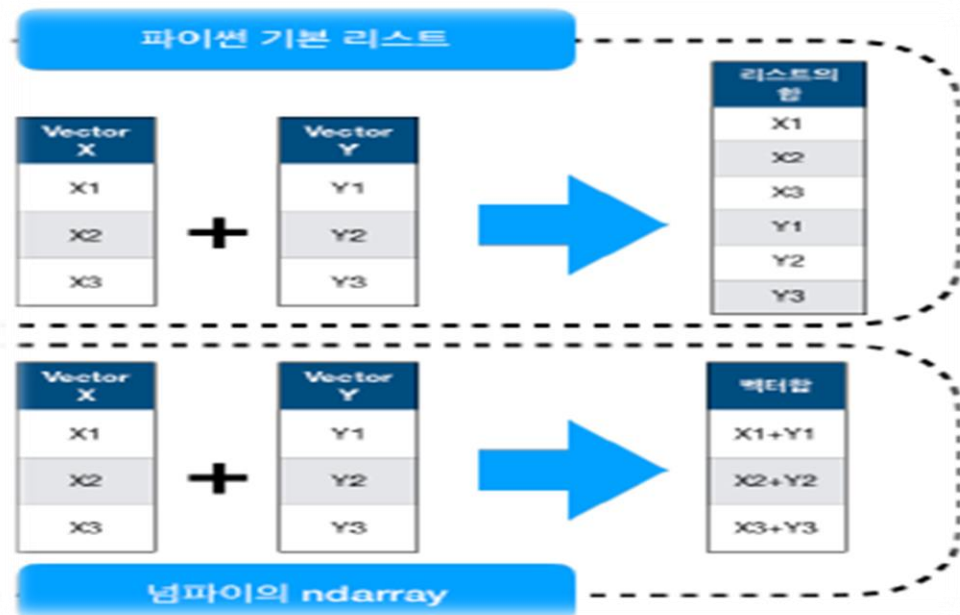
Numpy의 연산은 왜 빠를까?

- NumPy는 각 배열마다 타입이 하나만 있다고 가정 (아주 중요!!)
- 즉 배열안에 동일한 타입의 데이터만 저장 가능 (e.g. 정수면 정수, 실수면 실수만)
- 만약 여러 타입의 자료형을 사용하는 경우 NumPy에서는 문자열로 처리

```
>>> tangled = np.array([ 100, 'test', 3.0, False])
>>> print(tangled)
['100' 'test' '3.0' 'False']
```

```
mid_scores = [10, 20, 30]
final_scores = [70, 80, 90]

total = [ ]
for i in range(len(mid_scores)):
    total.append(mid_scores[i] + final_scores[i])
total
[80, 100, 120]
```



Numpy 배열 연산

① Numpy 패키징 / array 객체 연산

```
import numpy as np
```

- NumPy의 array() 함수를 이용한 배열 생성

```
mid_scores = np.array([10, 20, 30])
final_scores = np.array([60, 70, 80])
```

- NumPy array 객체 연산

<i>mid_scores</i>	10	20	30
	+		
<i>final_scores</i>	60	70	80
<i>total</i>	70	90	110

```
total = mid_scores + final_scores
print('시험성적의 합계 :', total)    # 각 요소별 합계가 나타난다
print('시험성적의 평균 :', total/2) # 모든 요소를 2로 나눈다
```

```
시험성적의 합계 : [ 70  90 110]
시험성적의 평균 : [35. 45. 55.]
```